

Embedding Engine — Strategy Pattern cho Multi-Engine

Tài liệu giải thích kiến trúc Strategy Pattern được áp dụng trong tầng Embedding, cho phép hệ thống chuyển đổi giữa Local Engine (miễn phí, offline) và OpenAI Engine (trả phí, chất lượng cao) chỉ bằng 1 biến môi trường.

Tham chiếu mã nguồn: [src/retriever/engines.py](#)

1. Strategy Pattern là gì?

Strategy Pattern là một mẫu thiết kế hành vi (behavioral design pattern) cho phép định nghĩa một **họ thuật toán** (family of algorithms), đóng gói từng thuật toán thành class riêng, và cho phép chúng **hoán đổi** cho nhau tại runtime mà không ảnh hưởng đến code sử dụng.

Trong hệ thống DentalAI, "thuật toán" ở đây là cách chuyển văn bản thành vector (embedding). Hai chiến lược (strategy) được hỗ trợ:

Engine	Model	Số chiều	Chi phí	Yêu cầu
local	keepitreal/vietnamese-sbert	768	Miễn phí	~420 MB RAM
openai	text-embedding-3-small	1536	~\$0.02/1M tokens	OPENAI_API_KEY

2. Cấu trúc class

```
EmbeddingEngine (ABC)          ← Interface trừu tượng
├─ embed_query(str) → ndarray   ← Embed 1 câu
├─ embed_batch(list) → ndarray ← Embed hàng loạt
├─ dimension → int             ← Số chiều vector
├─ name → str                  ← Tên engine

OpenAIEngine(EmbeddingEngine)   ← Strategy 1: API cloud
├─ Gọi OpenAI Embeddings API, batch_size=100, delay 0.3s chống rate limit

LocalEngine(EmbeddingEngine)    ← Strategy 2: Offline local
├─ sentence-transformers, batch_size=64, normalize_embeddings=True

create_engine(name) → Engine    ← Factory function
├─ Registry dict: {"openai": OpenAIEngine, "local": LocalEngine}
```

Abstract Base Class

EmbeddingEngine là interface trừu tượng (ABC) định nghĩa 4 phương thức bắt buộc mà mọi engine phải triển khai:

- `embed_query(query: str) → np.ndarray`: embed 1 câu query, trả về vector float32 shape `(dim,)`
- `embed_batch(texts: list[str]) → np.ndarray`: embed batch, trả về matrix float32 shape `(N, dim)`
- `dimension → int`: số chiều vector đầu ra (768 hoặc 1536)
- `name → str`: tên engine, dùng làm tên thư mục lưu FAISS index (`vector_db/{name}/`)

Factory Function

`create_engine(engine_name)` tra cứu `_ENGINE_REGISTRY` dict và khởi tạo class tương ứng. Nếu tên không hợp lệ, raise `ValueError` với danh sách engine hợp lệ.

3. Chi tiết từng Engine

3.1. LocalEngine (vietnamese-sbert)

Model: `keepitreal/vietnamese-sbert` — model sentence-transformers được huấn luyện riêng cho tiếng Việt, dựa trên PhoBERT.

Đặc điểm:

- Chạy hoàn toàn offline sau lần tải đầu tiên (~420 MB về `~/.cache/huggingface`)
- Singleton pattern qua `@lru_cache(maxsize=1)` — model chỉ load 1 lần dù tạo nhiều instance
- `normalize_embeddings=True` — vector output đã L2-normalized, tương thích IndexFlatIP
- Tắt auto-conversion safetensors bằng biến môi trường `DISABLE_SAFETENSORS_CONVERSION=1` để tránh lỗi offline
- Fallback: ưu tiên `local_files_only=True`, nếu model chưa có thì tải từ HuggingFace

3.2. OpenAIEngine (text-embedding-3-small)

Model: `text-embedding-3-small` — model embedding của OpenAI, 1536 chiều.

Đặc điểm:

- Gọi API qua `openai.embeddings.create()`
 - Batch embedding: tối đa 2048 inputs/request, dùng `batch_size=100` để an toàn với rate limit Tier-1
 - Delay 0.3s giữa các batch để tránh rate limit
 - Sort response theo `index` để đảm bảo thứ tự khớp với input
 - Vector output đã được OpenAI normalize sẵn
-

4. Cách chuyển đổi Engine

Chỉ cần thay đổi 1 biến trong file `.env`:

```
EMBEDDING_ENGINE=local    # Dùng vietnamese-sbert (miễn phí)
EMBEDDING_ENGINE=openai   # Dùng text-embedding-3-small (trả phí)
```

Sau đó chạy lại ingest để tạo FAISS index mới:

```
python -m src.retriever.ingest --engine local
# hoặc
python -m src.retriever.ingest --engine openai
```

FAISS index được lưu tách biệt theo engine (`vector_db/local/` và `vector_db/openai/`), nên có thể tồn tại song song.

5. Tại sao dùng Strategy Pattern?

Lợi ích	Giải thích
Open/Closed Principle	Thêm engine mới (VD: Cohere, Gemini) chỉ cần tạo class mới + thêm vào registry, không sửa code hiện tại
Dễ test	Mock engine trong unit test mà không cần API key
Dễ so sánh	Chạy ingest cho cả 2 engine, so sánh chất lượng retrieval trên cùng dataset
Linh hoạt triển khai	Dev dùng local (miễn phí), production dùng OpenAI (chất lượng cao)